MIE1517 Sample Midterm Questions

Part 1: Introduction

"Training": (whether you "know" the material)

1. What is the difference between artificial intelligence, machine learning, and deep learning?
   Answer: AI: create intelligent machines that work and act like humans; ML: apply an algorithm that automatically learns from example data; DL: using deep neural networks to automatically learn from example data.
2. What is the difference between supervised and unsupervised learning?
   Answer: Supervised learning: learning a model that maps an input to an output based on example input-output pairs. Unsupervised learning: learning the structure of some (unlabeled) data.
3. What is the difference between regression and classification?
   Answer: Regression and Classification problems both belong to supervised learning. For regression, the output is a continuous value, while for classification, the output is categorical value.
4. A neuron has a cell body, axons, and dendrites. What order does information flow through these parts?
   Answer: Dendrites->cell body (soma)->Axon.
5. What is a synapse?
   Answer: the area where the axon of one neuron and the dendrite of the other connect
6. What does "feed-forward" mean in the context of an artificial neural network?
   Answer: the information moves in only one direction, forward, from the input nodes through the hidden nodes and to the output nodes. There are no cycles or loops in the network.
7. What does "fully-connect" mean in the context of an artificial neural network?
   Answer: each neuron in one layer is connected to all neurons in the next layer
8. A loss function takes two arguments. What are they?
   Answer: model prediction and actual (ground truth) data
9. Which of the following are ~~*not*~~ steps in neural network training?
   - computing the prediction (Yes)
   - computing the loss function (Yes)
   - resetting the parameters to random (No, only when initialized but does not have to be random)
   - displaying the image (Not required, but could included)
10. Why do we need both a training set and a test set?
    Answer: The training set is required for the model to learn patterns in the data, the validation set is used to verify that the model can generalize.
11. Gradient descent is an example of . . .

- a loss function
- an optimizer (True)
- a neural network architecture
- a type of neuron unit

Generalization: (whether you can apply the material)

1. A researcher wishes to train a neural network to play the game Mario. Is this problem an example of supervised, unsupervised, or reinforcement learning?

   Answer: Reinforcement Learning

2. A researcher wishes to train a neural network to play the game Mario. The researcher rewarded the network based on the amount of time (in seconds) that Mario stayed alive. Would you expect the network to perform well? Why or why not?

   Answer: No, Mario might end up staying at the same place

3. How is an artificial neural network similar to a biological neural network? How are they different?

   Answer: Similarities include: there is only one direction of information flow, activation is similar to action potential, cells in one layer contribute differently to the same cell in the next layer. Differences: biological neural networks are not fully connected, they can encode information through firing frequency, greater complexity at all scales.

4. Consider the forward function of a neural network defined using PyTorch:

```
def forward(self, img):
    return self.layer5(img.view(-1, 256 * 256))
```

   - What is the expect size of the input image? 256*256
   - How many layers are in this network? 1 layer
   - How many hidden units are in this network? 0
   - How many output units are in this network? Not enough information

5. Which of the following are binary classification problems?
   - Finding traffic lights in an image Yes
   - Determining the sex of a duck based on its height, weight, and other features Yes
   - Determining the sex of a duck based on its photograph Yes
   - Determining the maker of a car based on a photograph (out of the major car makers) No
   - Translating from English to French No

6. Consider this training loop. Which of the following are true?

```
for i in range(50):

   for (input, actual) in data:
       out = network(input)
```

```
loss = criterion(out, actual)
loss.backward()
optim.step()
optim.zero_grad()
```

- The training data contains 50 items Not provided
- The training data contains 1 item Not provided
- Each training example is used once No
- Each training example is used 50 times Yes
- The optimizer used is gradient descent Not provided with exact optimizer
- The network is a 3-layer neural network Not provided
- The variable name of the neural network is pigeon No
- The variable name of the neural network is network Yes
- The variable name of the neural network is input No

Part 2: Model Training

"Training": (whether you "know" the material)

1. Sketch the ReLU, sigmoid, and tanh activation functions. See ANN lecture slides
2. When training a neural network on a binary classification problem. Is it possible for the network's training accuracy to stay the same even while the loss function decreases?

   Answer: Yes. Because training accuracy is discrete while loss is continuous.
3. Why do we not count the input layer when we count the number of layers in a neural network?

   Answer: This is a convention. The purpose is to match the number of layers with weights/parameters to the number of layers in a neural network.
4. When training a neural network, what does the optimizer optimize? Does the optimizer optimize the *same* quantity in each iteration, or a different one? Explain.

   Answer: The optimizer optimizes the parameters of the network such that the loss is minimized by the network given the batch data. No. The batch data is different in each iteration so the loss function is different for each iteration.
5. What can happen when a batch size is too small? Too large?

   Answer: Too small has more noisy, faster to train, optimize very different loss function for each iteration.

   Too large: more representative of the data and less noisy, more computation.
6. How is a training curve helpful in detecting overfitting?

Answer: We use a separate data set, validation set, to detect overfitting. If the training error keeps decreasing while the validation error starts to increase, overfitting occurs. The key point is that the validation set needs to be separated from the training stage.

7. What are hyperparameters? What are some examples of hyperparameters?

   Answer: Hyperparameters are parameters that cannot be tuned by the optimizer. Examples: # of layers in a neural network, # of neurons in each layer, activation function, batch size, learning rate.

8. How do we tune hyperparamters?

   Answer: We tune hyperparameters based on the validation error. The goal is to minimize underfitting and overfitting on the training set while maintaining the smallest validation error.

9. Why is checkpointing important when you train a neural network?

   Answer: Robust to runtime errors, crashes, etc., could allow you to load previous model parameters based on learning curve.

10. Label the purpose of each line in the training code below:

```
for epoch in range(num_epochs): #loop over entire dataset = num_epochs
    for data, actual in iter(train_loader): #loop through all batches
        out = model(data) #model output/prediction
        loss = criterion(out, actual) #compute loss
        loss.backward() #compute gradients for parameters
        optimizer.step() #update the parameters
        optimizer.zero_grad()#zero out the gradients for next iteration
```

Generalization: (whether you can apply the material)

1. Bob proposes to use a function f(x) = min(x, 0). Will this activation function make the network easier to train or harder to train? (Hint: the ReLU activation can also be written as relu(x) = max(x, 0))

   Answer: similar

2. Suppose you have a dataset of 10000 labeled training data. If you choose a batch size of 50, how many iterations are in an epoch? What about if you choose a batch size of 100? a) 200, b) 100

3. You find that with a batch size of 64 and a learning rate of 0.0001, your neural network trains too slowly. What can you do?

   Answer: Increase the learning rate, decrease the number of batch size (less computationally expensive)

4. One “hyperparameter” we haven’t discussed is the initial (random) value of the parameters. Since neural network training does not find the *globally* optimal value of the parameters, the initial value of the parameters can be important. How can we tune this (set of) hyperparameters?

   Answer: Use the seed function to generate different sets of initial parameters and compare their performance based on the validation error.

5. John and Sally are both training neural networks to classify cats vs dogs. John tuned his hyperparameters by training 20 different neural network models and choosing the best one. Sally tuned her hyperparameters by training 2000 different neural network models and choosing the best one. For the following questions, assume that the validation and test sets are fairly large.
    - Whose model do you think will have the better validation accuracy? Sally
    - Would you expect Sally's test accuracy to be lower/higher than her validation accuracy? Lower
    - Would you expect John's test accuracy to be lower/higher than his validation accuracy? Lower
    - Consider the difference between the test accuracy and the validation accuracy. Would you expect this difference to be higher for Sally or for John? Sally

Part 3: Artificial Neural Networks

"Training": (whether you "know" the material)

1. What is the purpose of the softmax activation? How is it similar to the sigmoid activation? How is it different?

   Answer: They are both used in deep learning to calculate probability. Softmax activation accepts a set of vectors and return the probability distribution, i.e. all outputs sum up to 1. The sigmoid activation accepts a single input and return a probability (i.e a value between 0 and 1)
2. What is one technique to debug a neural network, to make sure that the programming is likely to be correct?

   Answer: display the size (shape) of the model tensors, or verify that your model can overfit to a small dataset.

Generalization: (whether you can apply the material)

1. Identify 3 issues with the neural network model below:

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.layer1 = nn.Linear(28 * 28, 40)
        self.layer2 = nn.Linear(30, 1)

    def forward(self, img):
        flattened = img.view(-1, 28 * 28)
        activation1 = self.layer1(flattened)
        activation2 = self.layer2(activation1)
        return torch.sigmoid(activation2)
```

Answer: The number of neurons (40) in the first hidden layer does not match the number of neurons (30) in the second hidden layer. Should use activation function in intermediate layers. Should not use sigmoid activation on the output.

2. Normally, the sigmoid or softmax activation in the last layer of the neural network is **not** applied in the `forward()` method. Why is that? (You will need to do some research for this question. You're not expected to understand the mathematics behind the reasoning, only the reasoning itself)
   Answer: The computation of the loss function is more numerically stable when we don't run the softmax function (we get a more accurate loss function value because of the way floating-point values are represented on the computer).

Part 4: Convolutional Neural Networks

"Training": (whether you "know" the material)

1. What are the advantages of convolutional layers over fully-connected layers?
   Answers: Convolutional layers allow locally connected layers and weight sharing. For a large input image, using a fully-connected layer requires lots of parameters. It is computationally expensive. Convolutional layers allow you to tune fewer weights, making the training process faster. Convolutional layers can take in arbitrary sized inputs.
2. PyTorch convolutions expect data in the "NCHW" format. What does this mean?
   Answer: N: number (batch size), C: channel, H: height, W: width
3. Why does it make sense to share weights in a convolutional neural network?
   Answer: Same weights allow for extracting the same feature over different spatial regions in the image.
4. What is the purpose of zero padding in a convolutional layer?
   Answer: Keep the information around the border of the image, and provides a way to keep the next layer width and height dimensions consistent with the previous layer.
5. Why should we design our neural networks so that the number of activations in each layer generally decreases?
   Answer: We want the network to consolidate information and throw out useless data.
6. Explain the idea of transfer learning, specifically of using AlexNet features to train a neural network more quickly.
   Answer: Transfer learning allows you to take advantage of a pretrained feature extractor learned from a dataset with millions of samples. The pretrained convolutional neural network is then combined with fully-connected layers to produce an output for a new dataset. The new data is only used to fine tune the parameters of the fully-connected network. This offers significant speedups.
7. What is the output of the following code?

```
>>> x = torch.randn(20, 3, 16, 16) # NCHW
>>> conv = nn.Conv2d(in_channels=3, out_channels=7, kernel_size=5,
padding=0)
>>> conv(x).shape
```

Answer: `torch.size([20, 7, 12, 12])`

8. What new idea was introduced in GoogLeNet? ResNet?

   Answer: GooLeNet use repeated module & several very small convolutions in order to drastically reduce the number of parameters. ResNet uses skip connections to reduce vanishing gradients and allow for training of deeper network.

9. What are fully-connected networks? Name one advantage and one disadvantage to using a fully convolutional network.

   Answer: Neural networks that have a parameter/connection between all combinations of inputs and outputs.

Generalization: (whether you can apply the material)

1. How many multiplications are performed when applying a 3x3 convolution on a greyscale image of size 7x7?

   Answer: 5*5*3*3 = 225

2. Which operations to reduce the number of units in the hidden layer are the most and least computationally efficient: max pooling, average pooling, and using strides in a convolution?

   Answer: Most computationally efficient: strided conv since we're reducing the computation required in the computation layer and are not adding additional layers, Least computationally efficient: average pooling

3. Suppose we learn a fully-connected network on a 128x128 image. Can we use the same network to make predictions about images of size 256x256? Why or why not?

   Answer: No. Different input sizes, the images would have to be resized to work with a fully-connected neural network.

Part 5: Autoencoders

"Training": (whether you "know" the material)

1. Describe the idea of a "distributed" encoding.

   Answer: It is a way to encode data such that they are represented by different points in the embedding space. Similar data are closer in the embedding space.

2. What is data augmentation?

Answer: Data augmentation is a way to increase the size of the data set. This is used when we don't have enough data for training the model, or we want to introduce augmentations that may appear in new data. Data augmentation makes use of the invariance property of CNN networks (independent of translation, scaling, and rotation) and creates new data based on existing data. Normal ways of increasing dataset include rotating by a random angle, translation, flipping, scaling, cropping, and Gaussian noise.

3. Describe the following strategies for preventing overfitting: data augmentation, data normalization, model averaging, dropout, weight decay, and early stopping.
   Answer: Data augmentation: Increase data size by manipulating (rotating, translating, cropping scaling, flipping) the given data. Data normalization: Adjust input feature values to be scaled similarly. Model averaging: Build many models and average their behaviours. Dropout: Randomly remove a portion of neurons from each iteration. Weight decay: The idea of weight decay is to penalize large weights by adding a term (sum of wk, sum of wk^2) to the loss function. Large weights are not ideal because it means the prediction relies a lot on the content of one feature. Early stopping: Reduce the number of training epochs as the validation error begins to increase to prevent overfitting to the training data.
4. In PyTorch, why is it important to mark whether a model is currently used for training or for testing?
   Answer: Important to set model.train() or model.eval() to change the behavior of the dropout layer. The dropout layer behaves differently for training and testing.
5. Describe, intuitively, why large weights are indicative of overfitting.
   Answer: Large weights mean that the prediction relies a lot on the content of select features.
6. What is an autoencoder?
   Answer: Autoencoder is a type of generative model that consists of an encoder and a decoder and is trained using unsupervised learning. The decoder produces a low dimensional representation of the data and the decoder takes the low dimensional representation to produce a reconstruction of the original data.
7. What is the loss function used when training an autoencoder?
   Answer: Typically uses mean squared loss
8. What is an "embedding"?
   Answer: a lower dimensional space representing the data.

Generalization: (whether you can apply the material)

1. Consider augmenting an image dataset by shifting the training images by a random (small) number of pixels. For which type of neural network would this type of data augmentation have a bigger impact: a convolutional network or a fully-connected network?
   Answer: Fully-connected network. It does not remember features. Does not have the invariant property.

2. Suppose you have two networks: network A has 10 million hidden units (artificial neurons) and 1 million weights, network B has 10 million weights and 1 million hidden units. Which of the two networks has higher capacity? Which of the two networks is more likely to overfit?
   Answer: Network B has a higher capacity and is more likely to overfit.
3. In PyTorch, the implementation of weight decay in an optimizer (e.g. `optim.SGD(model.parameters(), weight_decay=0.0001)`) performs weight decay for all parameters, including biases. Why might we want to allow large biases, and only decay neural network weights?
   Answer: Biases provide a constant that is applied across the network and are not tied directly to a single input.
4. We discussed data augmentation for images. Suppose that you are instead working with voice recordings. What are some ways to augment a dataset of voice recordings?
   Answers: Add various types of noise, shifting time, changing pitch and speed.
5. What do you think would happen if the encoder output of an autoencoder is larger than the image size? You may assume that both the encoder and decoder have high capacity.
   Answers: The encoder may duplicate the sample input with no reduction in dimensionality.
6. What do you think would happen if the encoder output of an autoencoder is too small?
   Answer: More difficult to decode. All images will be similar in the embedding space and after reconstruction.
7. What does overfitting look like in the context of an autoencoder?

   Answer: The extreme end of overfitting in an autoencoder would simply be memorizing the data and returns a one-dimensional embedding space where the model has learned to an id code for each sample and can reproduce the samples based on that id. In this case the model has not learned anything meaningful.

8. Why do we use the sigmoid activation in an autoencoder of images?
   Answer: All of our image pixels are in the range [0, 1]. Sigmoid gives us results in the same range.
9. Suppose that an autoencoder used only convolutional layers, with no fully connected layers, and also no global pooling layers. Can the autoencoder be used to generate images of different sizes? If so how?
   Answer: Yes. Convolutional layers are independent of image size.